Лабораторная работа № 2

Дисциплина: Прикладное программирование

Тема: Регулярные выражения. Unit-тестирование

Вариант № 16: Проверка и поиск синтаксически корректного времени в формате ЧЧ:ММ:СС.

1. Цель работы

Изучить регулярные выражения в Python, реализовать проверку и поиск времени, написать unit-тесты для проверки программы и использовать Git для хранения проекта.

2. Постановка задачи

Программа должна проверять время, введенное пользователем, и искать записи времени в текстовом файле. Для проверки используется модуль `re`, для тестов — `unittest`. По условию можно выбрать поиск на веб-странице или в файле. В работе реализован поиск в файле.

Правила формата:

- часы от 00 до 23;

- минуты и секунды от 00 до 59;

- в каждой части ровно две цифры от 0 до 9;

- части разделены двоеточиями;

- при проверке строки лишние символы и пробелы недопустимы;

- при поиске рядом со временем не должно быть букв, цифр, подчеркивания или двоеточия.

Программа проверяет обычное время суток без часового пояса и дополнительных секунд. Например, `23:59:59` допустимо, `24:00:00` и `23:59:60` недопустимы.

3. Регулярное выражение

```
(?:[01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]
```

`[01][0-9]` задает часы от 00 до 19, а `2[0-3]` — от 20 до 23. Символ `|` означает выбор одного из двух вариантов. Группа `(?:...)` объединяет варианты, но не сохраняет их отдельно в результате поиска. Каждая часть `[0-5][0-9]` задает число от 00 до 59. Двоеточия обозначают обычные символы-разделители.

Для проверки используется `fullmatch()`, поэтому должна совпасть вся строка. Для поиска применяется выражение с дополнительными границами:

```
(?<![\w:])(?:[01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9](?![\w:])
```

Проверки `(?<![\w:])` и `(?![\w:])` запрещают соответствующие символы слева и справа. В Python `\w` включает буквы, цифры и подчеркивание. Это не дает найти `09:00:00` внутри `109:00:00` или `abc09:00:00`. Пробелы, скобки, запятые и точки могут отделять время от другого текста. Сами границы в найденный результат не входят.

4. Реализация

В `time_validator.py` находятся три функции:

1. `is_valid_time(value)` проверяет строку и возвращает `True` или `False`.

2. `find_times(text)` возвращает список найденных записей по порядку, сохраняя повторы.

3. `find_in_file(filename)` читает файл и передает его содержимое функции поиска.

Файл открывается через `with open(...)`, поэтому закрывается автоматически. Кодировка `utf-8-sig` позволяет читать UTF-8 как с BOM, так и без него.

В `main.py` реализовано меню с циклом `while` и выбором действия через `if/elif`. При ошибке открытия или неправильной кодировке выводится сообщение, после чего пользователь может выбрать другое действие. Доступны проверка строки, поиск в файле и выход. Сторонние библиотеки не используются.

5. Интерфейс и примеры выполнения

Запуск из папки проекта:

```
python main.py
```

Проверка корректной строки:

```
1. Проверить время
2. Найти время в файле
0. Выход
Выберите действие: 1
Введите время в формате ЧЧ:ММ:СС: 09:30:05
Время записано правильно.
```

Проверка ошибочной строки:

```
Выберите действие: 1
Введите время в формате ЧЧ:ММ:СС: 24:00:00
Неверное время. Пример правильной записи: 09:30:05.
```

Поиск в приложенном файле:

```
Выберите действие: 2
Введите путь к текстовому файлу: examples/schedule.txt
Найдено записей: 5
09:00:00
10:30:00
10:40:00
12:10:00
09:00:00
```

Ошибочные записи и части более длинных записей из этого файла не выводятся. Если в файле нет совпадений, выводится сообщение «Время в файле не найдено».

6. Тестирование

Тесты находятся в `tests/test_time_validator.py`. Для запуска используется команда:

```
python -m unittest discover -s tests -v
```

Написано 17 тестовых методов. Внутри части методов используется `subTest()`, чтобы проверить несколько входных значений отдельно. Для часов проверены все двузначные значения от 00 до 99. Аналогично проверены минуты и секунды. Также проверяются пустые строки, неправильные разделители, неполные записи, лишние пробелы и цифры других алфавитов.

Для поиска проверяются порядок результатов, повторы, разделение знаками препинания, отсутствие совпадений и запрет на извлечение части другой записи. Для файлов проверяются UTF-8, BOM, пустой и отсутствующий файл, неверная кодировка. Временные файлы создаются через `TemporaryDirectory()` и удаляются автоматически. Меню проверяется с подменой ввода через `unittest.mock.patch()`: проверяются выбор пунктов, результаты поиска, ошибки и возвращение в меню.

Результат запуска: **17 тестов выполнены успешно, ошибок нет**.

7. Вывод

В ходе работы составлено регулярное выражение для времени в формате ЧЧ:ММ:СС. Реализованы проверка пользовательского ввода и поиск в текстовом файле. Unit-тесты проверяют диапазоны значений, поиск и обработку ошибок. Код и отчет сохранены в Git.

Репозиторий: https://github.com/s1qtewenskii/library-lab-2